CERTIFY low-end RISCV node Software documentation

Introduction

The objective of this document is to present the software the CERTIFY low-end open hardware node is equipped with.

With regards to the CERTIFY architecture and requirements, these are the high level requirements for an IoT node to be able to support the services offered by the CERTIFY framework:

Boot integrity:

- The initial state of the device must correspond to a predetermined value.

Service operation:

- A remote authenticated user must be able to install[1], uninstall[2] and update remotely a service,
- A remote authenticated user must be able to start[3], stop[4] and restart remotely a service,
- A remote user must be able to communicate with a service,
- Two services must be able to communicate with each other.

Service isolation:

- The service code and private data regions must have integrity and isolation[5] guarantee.

Service verification:

- A remote authenticated user must be able to verify the integrity and authenticity of a service.

Behaviours control:

- A remote authenticated user must be able to change some or all of the authorisations granted to a service. (e.g. permit/prohibit the use of encryption or the use of secure storage),

[1] The service is downloaded into the permanent storage of the IoT node.
[2] The service is removed from the permanent storage of the IoT node.
[3] The service execution begins.
[4] The service execution stops.
[5] Only the service itself is able to access those regions.

- A remote authenticated user must be able to set a security policy that, if triggered, will automatically enable/disable a service authorisation.

Encryption:

- An authorised service must be able to request the generation of a random number,
- An authorised service must be able to request data digests,
- An authorised service must be able to request data authentication,
- An authorised service must be able to request symmetric encryption and decryption of data,
- An authorised service must be able to request the derivation of a cryptographic key.

Secure storage:

- An authorised service must be able to request the creation, destruction, reading and writing of a file on a permanent medium with the following characteristics:
    - Confidentiality: The content must be accessible only to the service itself,
    - Integrity & Authenticity: The service must be able to verify the integrity and the authenticity of the file content.

CERTIFY le-node: high level context

CERTIFY le-node is the name of the system. Following is a high level contextualisation of the subsystems.

The Secure Monitor (SM) is the subsystem that provides all essential software primitives to meet the low-level requirements. A Secure Enclave (SE) is the subsystem that provides the software “capsule” that contains the software routines of a CERTIFY service and whose code and data memory is isolated from any access other than by the SE itself. The First Stage Secure Bootloader (FSSBL) is the subsystem that provides the trusted bootloader programmed in the ROM of the device that is responsible for the secure boot of the SM component.

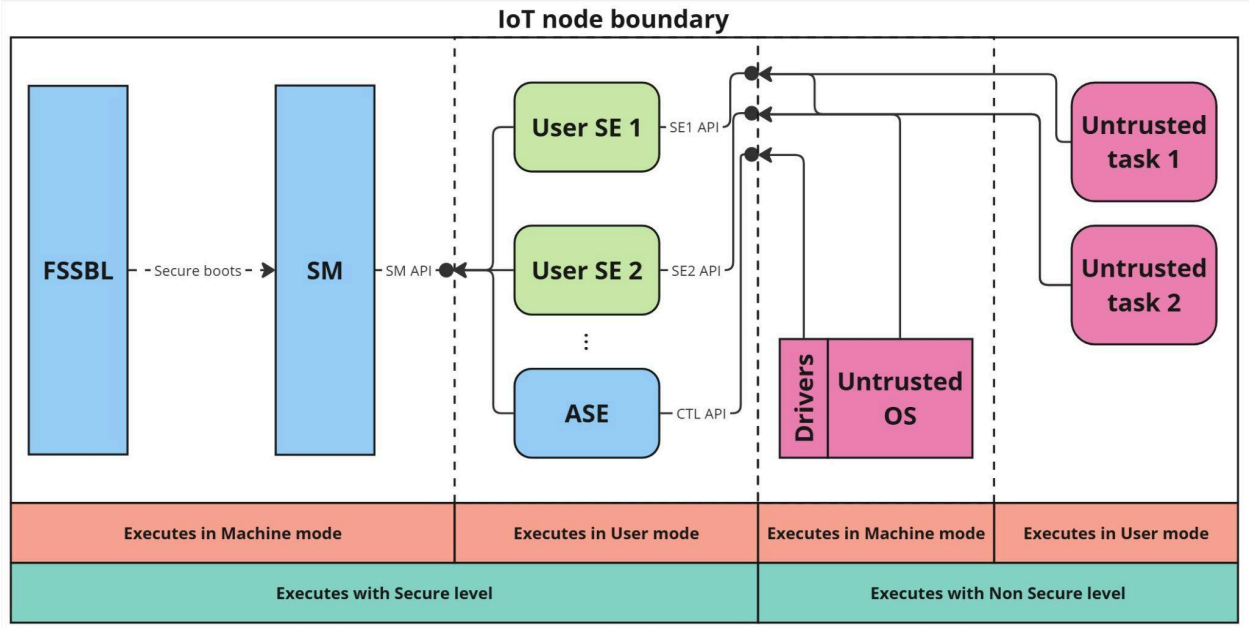
There is a special SE named **Architectural Secure Enclave** (ASE). This Enclave enables the interaction with the SM from outside the secure perimeter. Its significance lies in its ability to separate the SM core security mechanisms from user-level operations, such as installing a new SE on the device or deriving a domain key.

The Architectural Enclave is the only one fully authorised by the SM at startup, meaning it can request the SM to perform any sensitive actions. Otherwise, it behaves like any other SE, adhering to the same characteristics and functionalities.

The SEs issue requests to the SM through the SM API. Each user SE defines and implements a specific SE API through which it exposes its services. The ASE is in charge of defining and implementing the CTL API which is the only way to issue requests to the SM from outside.

Untrusted OS and Untrusted tasks are foreseen in the general scenario but their presence is optional. They can issue requests to the SM through the CTL API or ask a SE service through a specific SE API.

The following is a high level domain diagram showing the main subsystems and interfaces with their execution modes and security level; the different colours of the components serve to emphasise the different entities that created them.



FSSBL: Functional requirement specification

High level family	
Boot integrity	The FSSBL must copy the image of the SM from FLASH to OCM, compute the digest, check that the value matches to the one stored in ROM and then start the execution of the SM code giving it access to the device root key, MUD url and any relevant information stored in ROM.

SM: Functional requirement specification

High level family	
Service operation	The SM must allow an authorised SE to implant a new SE, meaning that the new SE digest golden value is calculated, saved permanently and returned to the caller.
	The SM must allow an authorised SE to remove an implanted SE, meaning that the digest golden value is removed from the permanent storage of the SM.
	The SM must allow an authorised SE to boot a SE image, meaning that the SM will check that the digest of the provided SE image matches any of the implanted ones and then it will create both an isolated memory environment and a shared memory one for the SE, set the provided authorisations, copy its image inside the isolated one, authenticate that memory with the key whose ID is provided and start its execution.
	The SM must allow an authorised SE to shutdown another SE, meaning it will stop the execution of the SE and deallocate its isolated and shared memory environment.
	The SM must allow an authorised SE to reboot another SE, meaning that, provided a new set of authorisations, the SM will overwrite the old ones and execute again the boot routine for that SE.
	The SM must allow an authorised SE to execute a software routine exposed by another SE.
Service verification	The SM must allow a SE to require an SM-authenticated measurement of itself and – optionally – the measure of the

	underlying software stack. It can request either the **cold value**, which was calculated once at startup of the SE, or a **hot value**, which is recalculated at the moment of the request. A SE can request ONLY its authenticated measure regardless of its authorisations.
Behaviours control	The SM must store a log of all the requests issued by SEs. Each time a SE makes a request, the SM logs the issuer, the request type and if it was successful.
	The SM must allow an authorised SE to access the log file in read only mode.
	The SM must allow an authorised SE to require the change of one or more authorizations of another SE.
	The SM must allow an authorised SE to install a trigger that, if triggered, will automatically change one or more authorizations of another SE.
Secure storage	The SM must allow an authorised SE to create a file in the secure storage.
	The SM must allow an authorised SE to delete a file in the secure storage.
	The SM must allow an authorised SE to read a file in the secure storage.
	The SM must allow an authorised SE to write a file in the secure storage.
Encryption	The SM must allow an authorised SE to require the derivation of a cryptographic key providing the ID of the source key, a

	seed for the derivation and the new key ID. Any key is permanently stored by the SM with confidentiality and integrity guarantee.
	The SM must allow an authorised SE to delete a stored key. The only key that can never be removed is the device root key.
	The SM must allow an authorised SE to generate a random number.
	The SM must allow an authorised SE to require a data digests.
	The SM must allow an authorised SE to require data authentication providing the ID of a stored authentication key.
	The SM must allow an authorised SE to require symmetric encryption of data providing the ID of a stored encryption key.
	The SM must allow an authorised SE to require symmetric decrypt data providing the ID of a stored encryption key.

SM: Non functional requirements specification

High level family	
Service isolation	The SM must ensure that, at all times, each SE isolated memory environment is accessible only by that SE.

SE: Functional requirements specification

High level family	

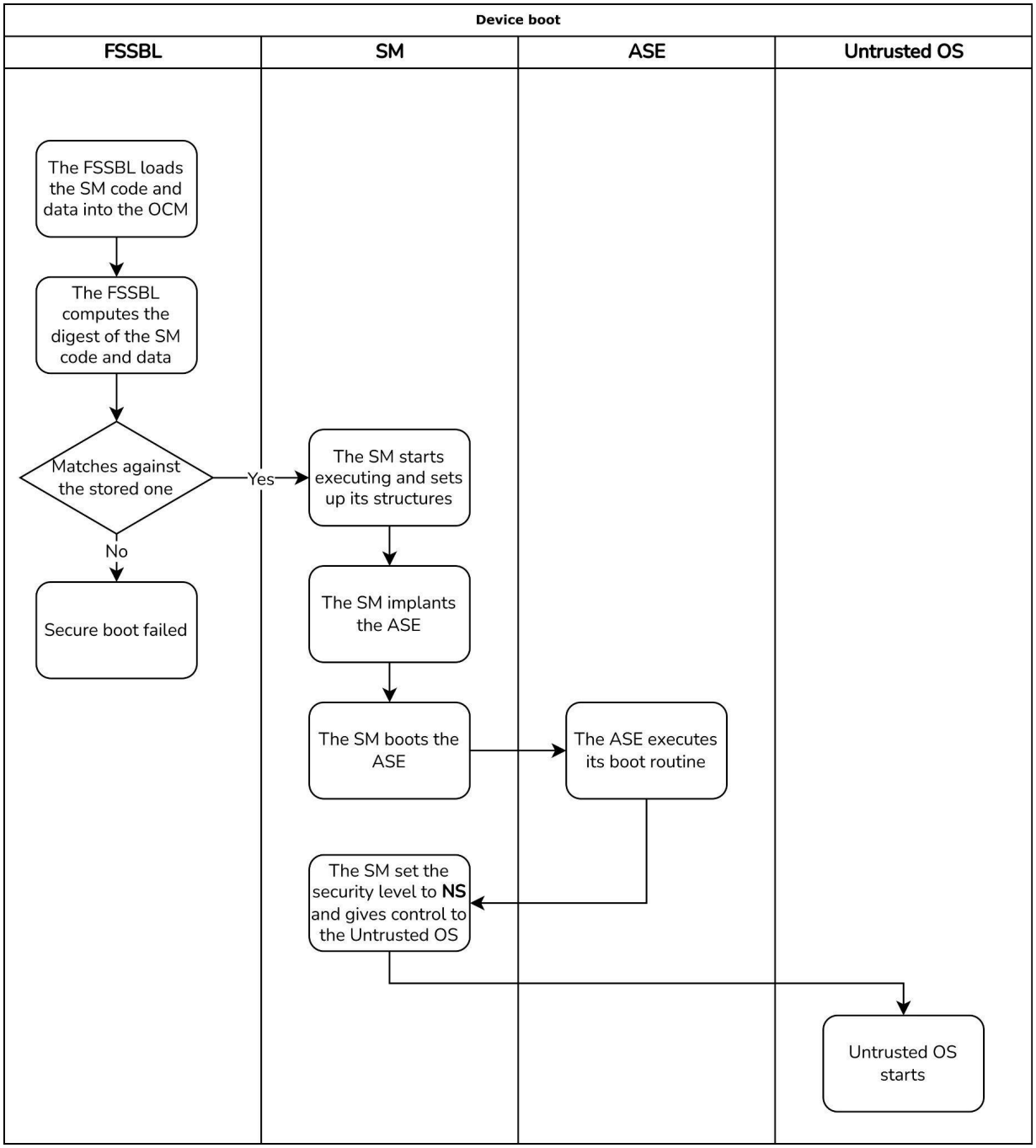
Service operation	The SE must expose an interface to the SM containing the boot routine and all the routine it wants to expose to the outside world.

Main scenarios of the system

Two main scenarios of the system are analysed to demonstrate how the dynamic interaction between components takes place.

Scenario ID:	Scenario-1
Scenario Title:	Device boot
Goal:	The device is booted, the ASE is up and running and the system is ready to take requests.
Pre-condition(s):	● Device root key, MUD url, SM digest golden value and FSSBL are programmed in the ROM, ● SM image is programmed in the FLASH, ● The SM is provided with both the ASE digest golden value and the ASE image, ● The device boots in Machine mode and with Secure level.
Normal flow of events:	● The device receives current and the core starts executing the FSSBL code located in the ROM, ● The FSSBL copies the code and data of the SM from the FLASH to the OCM, ● The FSSBL measures the digest of the SM image and checks it against the golden value, ● The execution continues from the entry address of the SM in OCM, ● The SM initialises all its structures, ● The SM implants the ASE, ● The SM boots the ASE,

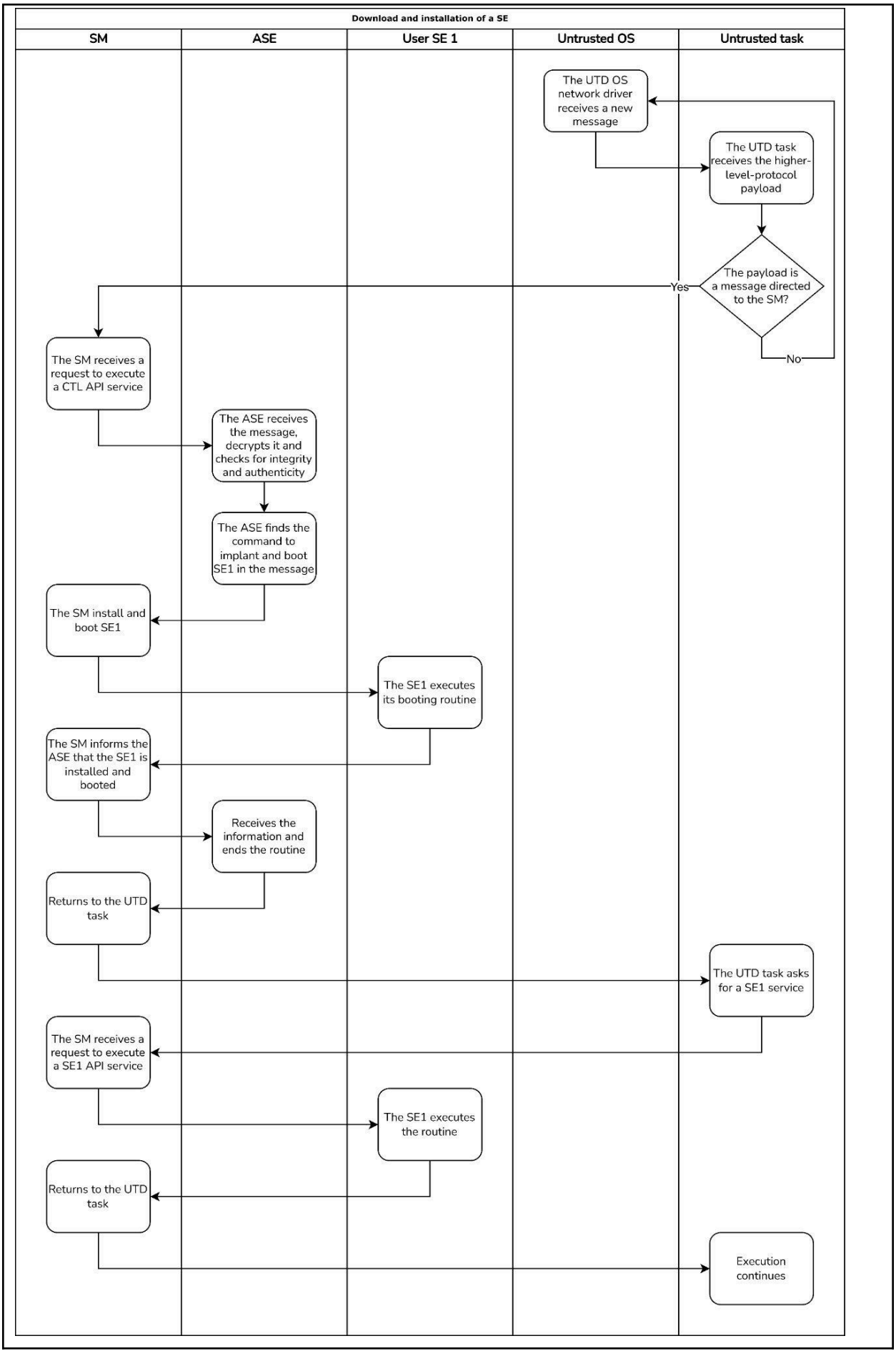
	• The SM sets the security level to **NS** and executes the Untrusted OS/App.



Post-condition:	• The ASE is up and running and ready to take requests, • The Untrusted OS/App is running in Machine mode and with NS security level.

Alternative flow(s) of events (under attack):	Not defined for now.

Scenario ID:	Scenario-2
Scenario Title:	Download, implantation, boot and usage of a SE
Goal:	The image of a SE is downloaded in the IoT node, implanted into the SM and booted. Then a request to one of its services is issued.
Pre-condition(s):	● The device is correctly booted and the ASE is up and running
Normal flow of events:	● The UTD OS network driver receives a message and deliver its higher-level protocol payload to a UTD task, ● The UTD task finds this is a control message to the SM and issues a request to process this message to the CTL API, ● The ASE decrypts the message and checks for integrity and authenticity, ● The ASE finds the message is a command to implant and boot a new SE (called SE1) and forwards these requests to the SM, ● The SM installs and boots SE1, ● The SM returns to the ASE the status of completion of the requests, ● The ASE returns the status of completion to the SM, ● The SM returns control to the UTD task forwarding the status of completion given by the ASE, ● The UTD task issues a request to the SE1 API, ● The SE1 executes the routine and returns the status of completion to the SM, ● The SM returns control to the UTD task forwarding the status of completion given by the SE1.


Download and installation of a SE
SM
ASE
User SE 1
Untrusted OS
Untrusted task
The UTD OS network driver receives a new message
The UTD task receives the higher-level-protocol payload
The payload is a message directed to the SM?
Yes
No
The SM receives a request to execute a CTL API service
The ASE receives the message, decrypts it and checks for integrity and authenticity
The ASE finds the command to implant and boot SE1 in the message
The SM install and boot SE1
The SE1 executes its booting routine
The SM informs the ASE that the SE1 is installed and booted
Receives the information and ends the routine
Returns to the UTD task
The UTD task asks for a SE1 service
The SM receives a request to execute a SE1 API service
The SE1 executes the routine
Returns to the UTD task
Execution continues

SM: Subsystem components

The following is a functional decomposition of the SM subsystem:

- Core: handles the system startup, other components startup, initial security configuration. Handles the S/NS context exchange and parameter security validation.
- Authorisation: check that a request is done with the sufficient authorisations.
- Tracer: store the request log, installation and execution of triggers.
- Enclave: implant, remove, boot, shutdown, reboot of SEs, execution of a SE routine from the outside, request from a SE of its authenticated measure, change a SE authorisation.
- Crypto: key derivation and secure storage, key removal, random num. gen., data digest, data authentication, symmetric encryption and decryption.
- Storage: create, delete, read and write to files with security properties both for SEs or for SM functionalities.